llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5 · claude-haiku-4-5-20251001 · BROKEN

−218 / +413 lines of 1967 · 51 parallel region(s) added · 17 of 41 candidate(s) kept · RESTRUCTURING (286 code line(s) added, 190 removed) · restructuring level 0

BEFORE

```
65                                const Index_t* elemToNode,
66                                Real_t elemX[8],
67                                Real_t elemY[8],
68                                Real_t elemZ[8])
69  {
70     Index_t nd0i = elemToNode[0] ;
71     Index_t nd1i = elemToNode[1] ;
72     Index_t nd2i = elemToNode[2] ;
73     Index_t nd3i = elemToNode[3] ;
74     Index_t nd4i = elemToNode[4] ;
75     Index_t nd5i = elemToNode[5] ;
76     Index_t nd6i = elemToNode[6] ;
77     Index_t nd7i = elemToNode[7] ;
78
79     elemX[0] = domain.x(nd0i);
80     elemX[1] = domain.x(nd1i);
81     elemX[2] = domain.x(nd2i);
82     elemX[3] = domain.x(nd3i);
83     elemX[4] = domain.x(nd4i);
84     elemX[5] = domain.x(nd5i);
85     elemX[6] = domain.x(nd6i);
86     elemX[7] = domain.x(nd7i);
87
88     elemY[0] = domain.y(nd0i);
89     elemY[1] = domain.y(nd1i);
90     elemY[2] = domain.y(nd2i);
91     elemY[3] = domain.y(nd3i);
92     elemY[4] = domain.y(nd4i);
93     elemY[5] = domain.y(nd5i);
94     elemY[6] = domain.y(nd6i);
95     elemY[7] = domain.y(nd7i);
96
97     elemZ[0] = domain.z(nd0i);
98     elemZ[1] = domain.z(nd1i);
99     elemZ[2] = domain.z(nd2i);
100    elemZ[3] = domain.z(nd3i);
101    elemZ[4] = domain.z(nd4i);
102    elemZ[5] = domain.z(nd5i);
103    elemZ[6] = domain.z(nd6i);
104    elemZ[7] = domain.z(nd7i);
105
106 }
107
108 static inline
109 void InitStressTermsForElems(Domain &domain,
110                              Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
…
303 static inline
304 void IntegrateStressForElems( Domain &domain,
305                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
306                               Real_t *determ, Index_t numElem, Index_t numNode
307 {
308
309    Real_t fx_local[8] ;
310    Real_t fy_local[8] ;
311    Real_t fz_local[8] ;
312
313   for( Index_t k=0 ; k<numElem ; ++k )
314   {
315     const Index_t* const elemToNode = domain.nodelist(k);
316     Real_t B[3][8] ;
317     Real_t x_local[8] ;
318     Real_t y_local[8] ;
319     Real_t z_local[8] ;
…
329     SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
330                                  fx_local, fy_local, fz_local ) ;
331
332     for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
333        Index_t gnode = elemToNode[lnode];
334        domain.fx(gnode) += fx_local[lnode];
335        domain.fy(gnode) += fy_local[lnode];
336        domain.fz(gnode) += fz_local[lnode];
337     }
338
339   }
340
…
495    gamma[3][4] = Real_t( 1.);
496    gamma[3][5] = Real_t(-1.);
497    gamma[3][6] = Real_t( 1.);
498    gamma[3][7] = Real_t(-1.);
499
500    for(Index_t i2=0;i2<numElem;++i2){
501       Real_t hgfx[8], hgfy[8], hgfz[8] ;
502
503       Real_t coefficient;
```

AFTER

```
65                                const Index_t* elemToNode,
66                                Real_t elemX[8],
67                                Real_t elemY[8],
68                                Real_t elemZ[8])
69  {
70     for (Index_t i = 0; i < 8; ++i) {
71        Index_t ndi = elemToNode[i];
72        elemX[i] = domain.x(ndi);
73        elemY[i] = domain.y(ndi);
74        elemZ[i] = domain.z(ndi);
75     }
76  }
77
78  static inline
79  void InitStressTermsForElems(Domain &domain,
80                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
…
273 static inline
274 void IntegrateStressForElems( Domain &domain,
275                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
276                               Real_t *determ, Index_t numElem, Index_t numNode
277 {
278   #pragma omp parallel for shared(domain, sigxx, sigyy, sigzz, determ) firstpr
279   for( Index_t k=0 ; k<numElem ; ++k )
280   {
281     Real_t fx_local[8] ;
282     Real_t fy_local[8] ;
283     Real_t fz_local[8] ;
284
285     const Index_t* const elemToNode = domain.nodelist(k);
286     Real_t B[3][8] ;
287     Real_t x_local[8] ;
288     Real_t y_local[8] ;
289     Real_t z_local[8] ;
…
299     SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
300                                  fx_local, fy_local, fz_local ) ;
301
302     for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
303        Index_t gnode = elemToNode[lnode];
304        #pragma omp atomic
305        domain.fx(gnode) += fx_local[lnode];
306        #pragma omp atomic
307        domain.fy(gnode) += fy_local[lnode];
308        #pragma omp atomic
309        domain.fz(gnode) += fz_local[lnode];
310     }
311
312   }
313
…
468    gamma[3][4] = Real_t( 1.);
469    gamma[3][5] = Real_t(-1.);
470    gamma[3][6] = Real_t( 1.);
471    gamma[3][7] = Real_t(-1.);
472
473    #pragma omp parallel for schedule(static, 128) shared(domain, gamma, determ
474    for(Index_t i2=0;i2<numElem;++i2){
475       Real_t hgfx[8], hgfy[8], hgfz[8] ;
476
477       Real_t coefficient;
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is